

Documentation détaillée de NovaEngine

NovaEngine est un moteur de jeu 2D moderne conçu pour être extensible et modulaire. Son architecture repose sur plusieurs piliers : un système **Entité-Composant-Système (ECS)** pour la logique de jeu, un **backend** abstrait pour l'interface avec la plateforme (fenêtre, rendu, audio, entrée, ressources...), un système d'**interface utilisateur (UI)** complet, un cœur applicatif léger et un gestionnaire de **scènes** permettant le voyage entre plusieurs mondes avec pathfinding. Le moteur propose également un **pipeline de post-traitements** pour appliquer des effets visuels comme le CRT, le bloom ou l'occlusion ambiante.

Cette documentation vise à fournir une vue d'ensemble exhaustive de ces modules et à illustrer leur utilisation avec des exemples de code inspirés des meilleures pratiques observées dans d'autres moteurs open source. Les exemples sont basés sur les API réelles décrites dans la documentation du moteur et ne doivent pas être considérés comme inventés.

1 Concepts de base et architecture

1.1 Architecture ECS

Le cœur de NovaEngine repose sur le modèle Entité-Composant-Système. Une **entité** est un identifiant sans logique qui peut posséder des **composants** attachés. Chaque composant est une structure de données décrivant un aspect de l'entité : position, sprite, lumière, animation, etc. Les **systèmes** parcourent le registre d'entités pour traiter les entités possédant certains composants. Cette approche favorise la flexibilité et la réutilisation : on compose des comportements en ajoutant ou retirant des composants.

Le moteur fournit une classe `EntityRegistry` pour gérer la création d'entités, l'ajout/retrait de composants et les requêtes. Le registre alloue les entités et garantit l'accès rapide aux composants. Les systèmes dérivent d'une classe de base `System` et surchargent une méthode `onUpdate` qui s'exécute chaque frame ¹. Les systèmes implémentés sont détaillés plus loin.

1.2 Backend abstrait

L'interface avec la plateforme est gérée par le **BackendManager**. Les macros `GRAPHICS()`, `AUDIO()`, `INPUT()` et autres fournissent un accès global aux sous-backends de rendu, d'audio, d'entrée et de ressources ². Cette abstraction permet de changer l'implémentation (par exemple SFML) sans modifier le reste du moteur. Chaque backend expose des interfaces :

- `IWindowBackend` pour créer la fenêtre et gérer les événements;
- `IGraphicsBackend` pour dessiner des textures, primitives ou quads;
- `IResourceBackend` pour charger les textures, fonts, sons et musiques;
- `IInputBackend` pour récupérer les touches et la souris;
- `IAudioBackend` et `IFontBackend` pour la lecture audio et le rendu de texte.

1.3 Système d'UI et cœur applicatif

Le système d'interface utilisateur repose sur une classe `UIComponent` de base et un `UIManager` responsable de l'ajout, de l'actualisation et du rendu des composants UI ³. Les composants UI existants incluent les boutons, labels de texte, images, panneaux, sliders, champs de texte et animations.

Le cœur applicatif se trouve dans la classe `Application`. L'utilisateur dérive de cette classe pour créer son jeu ; il doit surcharger les méthodes `onInitialize`, `onUpdate`, `onRender` et `onEvent` afin de définir le comportement spécifique de l'application. La méthode `run` du cœur gère la boucle principale : traitement des événements, mise à jour du moteur et rendu ⁴.

1.4 Gestion des scènes, définitions et pathfinding

Une **scène** représente un monde indépendant contenant ses entités et ses systèmes. Les scènes sont décrites dans des fichiers JSON placés dans `assets/data/scenes/`. Chaque scène possède un nom, une couleur de fond et une liste d'entités à instancier. Au lieu de dupliquer la configuration de chaque entité, NovaEngine adopte une **architecture à deux niveaux** : les **définitions** (niveau 1) décrivent des ressources réutilisables (sprites, animations, NPCs...) et sont chargées une seule fois; les **instances** (niveau 2) font référence à ces définitions via des identifiants et précisent leur position ou leurs paramètres ⁵. Par exemple, un sprite générique est défini dans `Sprites.json` et une scène fait référence à ce sprite via `"spriteID": "npc_merchant"`. Cette conception allège les fichiers de scène et centralise la modification des ressources.

Le `SceneManager` peut charger plusieurs scènes et basculer entre elles. On les charge via `SceneManager::loadScene("data/scenes/village.json", "village")` puis on active l'une d'elles avec `SceneManager::setActiveScene("village")` ⁶. Chaque scène peut déclarer des graphes de navigation locaux (waypoints) pour le pathfinding ainsi qu'un graphe global de transitions (`scenegraph.json`) pour connecter plusieurs scènes et permettre aux NPCs de voyager ⁷. Le **JourneySystem** planifie ensuite les déplacements d'une entité en calculant un chemin sur ce graphe et en déclenchant les transitions nécessaires ⁸.

Les fichiers de scène peuvent aussi spécifier la **couleur de fond** via un tableau `[R, G, B, A]`, ce qui permet de créer différentes ambiances : un bleu clair pour le jour ou un bleu foncé pour la nuit. Par exemple : `"backgroundColor": [135, 206, 235, 255]` représente un ciel d'été tandis que `"backgroundColor": [20, 20, 40, 255]` évoque la nuit ⁹.

1.5 Pipeline de post-traitement

Pour améliorer l'aspect visuel, NovaEngine intègre un **PostProcessPipeline**. Ce pipeline contient une liste d'effets (`PostProcessEffect`) appliqués successivement après le rendu de la scène. Les effets intégrés incluent :

- **CRTEffect** : simulation d'un affichage cathodique (CRT) avec distorsion et grain ¹⁰ ;
- **BloomEffect** : halo lumineux autour des objets brillants;
- **SSAOEffect** : occlusion ambiante en espace écran;
- **ColorGradingEffect** : correction de couleurs via une texture 3D (LUT);
- **DynamicLightingEffect** : simulation d'un éclairage dynamique 2D.

Le pipeline est modulaire : on peut ajouter, récupérer ou activer/désactiver des effets en cours d'exécution ¹⁰.

2 Composants disponibles

Voici un aperçu des composants fournis par NovaEngine. Les champs sont extraits directement de la documentation ; seuls les membres importants sont listés.

Composant	Description et champs principaux
TransformComponent	Définit la position (<code>position</code>), la rotation (<code>rotation</code>) et l'échelle (<code>scale</code>) d'une entité ¹¹ .
SpriteComponent	Contient une référence à une texture et un rectangle de texture (<code>rect</code>), plus l'ordre de dessin (<code>layer</code>) ¹² .
LightComponent	Représente une source de lumière 2D avec couleur, intensité, rayon et type (ponctuelle, directionnelle) ¹³ .
AnimationComponent	Gère une animation via un atlas de sprites, vitesse et boucle ¹⁴ .
ColliderComponent	Fournit une forme de collision (rectangulaire, circulaire) et des filtres pour la détection de collisions ¹⁵ .
AudioComponent	Permet d'attacher un son ou une musique à une entité et de contrôler la lecture (volume, boucle) ¹⁶ .
ActivatorComponent	Crée un déclencheur pour lancer une action lorsque l'entité interagit (par exemple, ouvrir une porte) ¹⁷ .
TagComponent	Stocke des chaînes identifiant l'entité, utile pour filtrer des entités ¹⁸ .
SceneTransitionComponent	Indique qu'une entité est une porte ou un passage vers une autre scène, avec la destination et le point d'entrée ¹⁹ .
JourneyComponent	Contient la progression d'un voyage inter-scènes : l'objectif, la scène de destination et l'état du trajet ²⁰ .

Exemple : création d'une entité avec composants

Le snippet ci-dessous montre comment créer une entité dans une scène et lui attacher un composant de transformation et un sprite :

```
// Création d'une entité via le registre de la scène
auto& registry = scene->getEntityRegistry();
Entity player = registry.create();

// Ajout de composants
auto& transform = registry.add<TransformComponent>(player);
transform.position = {0.f, 0.f};
transform.rotation = 0.f;
transform.scale = {1.f, 1.f};

auto& sprite = registry.add<SpriteComponent>(player);
sprite.texture = ResourceManager::getTexture("player.png");
```

```

sprite.rect = IntRect{0, 0, 64, 64};
sprite.layer = 1;

// Ajout d'un composant audio
auto& audio = registry.add<AudioComponent>(player);
audio.sound = ResourceManager::getSound("jump.wav");
audio.loop = false;

// L'entité apparaît maintenant dans le monde avec ses composants

```

Cet exemple s'appuie sur la création d'entités et l'ajout de composants tels que décrits dans la documentation : les composants sont simplement des données et peuvent être ajoutés dynamiquement ²¹.

3 Systèmes et logique de jeu

Les **systèmes** sont chargés de traiter les entités possédant certaines combinaisons de composants. Chaque système dérive de la classe `System` et implémente la méthode `onUpdate` appelée à chaque frame ¹. Les systèmes pré-intégrés incluent :

- **RenderSystem** : dessine les sprites et les lumières en tenant compte de la caméra et de la profondeur ²² ;
- **AnimationSystem** : met à jour les composants d'animation et change les images affichées;
- **LightSystem** : collecte les lumières et met à jour le rendu dynamique;
- **AudioSystem** : joue et stoppe les sons et musiques attachés aux entités;
- **PhysicsSystem** : gère les collisions et la physique simplifiée;
- **ActivatorSystem** : détecte l'activation de triggers et exécute des scripts ou événements ¹⁷ ;
- **JourneySystem** : orchestre le déplacement d'une entité entre plusieurs scènes en utilisant le graphe de scènes ⁸.

3.1 Créer un système personnalisé

Supposons que nous voulions un système pour réduire progressivement le volume des musiques pendant une scène. Nous pouvons créer une classe dérivée de `System` :

```

class FadeOutMusicSystem : public System {
public:
    void onUpdate(Scene& scene, float dt) override {
        auto& registry = scene.getEntityRegistry();
        // Sélectionne toutes les entités ayant un composant Audio
        registry.forEach<AudioComponent>([&](Entity entity, AudioComponent&
audio) {
            if (audio.music) {
                // Réduit le volume progressivement
                audio.volume = std::max(0.f, audio.volume - dt * 10.f);
            }
        });
    }
};

```

```
// Ajout du système dans la scène
scene.addSystem<FadeOutMusicSystem>();
```

L'exemple s'inspire du fonctionnement des systèmes décrits : la méthode `forEach` du registre permet d'itérer efficacement sur toutes les entités possédant un composant donné ¹.

3.2 Utiliser des activateurs

Les activateurs permettent de déclencher une action lorsque le joueur ou un objet interagit avec l'entité. Le composant `ActivatorComponent` contient un champ `onActivate`, par exemple un identifiant de script ou une clé d'événement ¹⁷. Voici comment créer un bouton ouvrant une porte :

```
// Crée un interrupteur dans la scène
Entity switchEntity = registry.create();
registry.add<TransformComponent>(switchEntity).position = {10.f, 5.f};
registry.add<SpriteComponent>(switchEntity).texture =
ResourceManager::getTexture("switch.png");
auto& activator = registry.add<ActivatorComponent>(switchEntity);
activator.onActivate = "open_door";

// Dans ActivatorSystem, lorsqu'une entité activable est touchée par le
joueur, on émettra un événement
```

Le système `ActivatorSystem` se charge ensuite de détecter l'activation (ex. collision avec le joueur) et d'envoyer l'événement `open_door` dans l'`EventDispatcher` ¹⁷.

3.3 Voyage inter-scènes (Journey)

Pour faire voyager un personnage non joueur entre plusieurs scènes, on utilise le `JourneyComponent` et le `JourneySystem`. Le composant stocke la scène cible, la destination et l'état actuel du voyage ²⁰. Lorsqu'une entité reçoit un voyage, le `JourneySystem` calcule un chemin sur le graphe de scènes (`SceneGraph`) puis invoque des transitions de scène et de waypoint ⁸.

```
// Attribue un voyage à un PNJ
Entity npc = ...;
auto& journey = registry.add<JourneyComponent>(npc);
journey.targetScene = "village";
journey.destination = {15.f, 8.f};
journey.state = JourneyState::Travelling;

// Ajoutez le JourneySystem à la scène si nécessaire
scene.addSystem<JourneySystem>();
```

Le système gère ensuite les transitions et appelle le `SceneManager` pour charger ou décharger les scènes selon le chemin à parcourir ⁸.

3.4 Composants personnalisés et système de dialogue

Le moteur est pensé pour être extensible : il est possible de créer des composants supplémentaires qui ne font pas partie du SDK de base. L'exemple Simple RPG fourni dans le dépôt ajoute un `DialogueComponent` afin d'attacher un nom et une liste de répliques aux NPCs. Ce composant n'est pas inclus dans la librairie principale et est défini dans le code de l'exemple ²³. Voici comment créer un composant personnalisé :

```
// Déclaration d'un composant Dialogue
class DialogueComponent : public Component {
public:
    std::string npcName;
    std::vector<std::string> dialogueLines;
    int currentLine = 0;
    // Macro pour attribuer un identifiant de type
    COMPONENT_TYPE_ID(DialogueComponent)

    void serialize(nlohmann::json& json) const override {
        json["npcName"] = npcName;
        json["dialogueLines"] = dialogueLines;
        json["currentLine"] = currentLine;
    }
    void deserialize(const nlohmann::json& json) override {
        npcName = json.at("npcName");
        dialogueLines =
        json.at("dialogueLines").get<std::vector<std::string>>();
        currentLine = json.value("currentLine", 0);
    }
};

// Utilisation : ajouter DialogueComponent à un NPC
auto& dialogue = registry.add<DialogueComponent>(npc);
dialogue.npcName = "Marcus";
dialogue.dialogueLines = {"Welcome traveler!", "What can I do for you?"};
```

Pour gérer ces dialogues, un système personnalisé `DialogueSystem` peut écouter l'entrée de l'utilisateur (touche E) et avancer dans le dialogue. Le système affiche une boîte de dialogue à l'écran pendant que le joueur est bloqué ²⁴. Un `PlayerController` personnalisé détecte les NPCs à proximité et transmet la commande au `DialogueSystem` ²⁵. Ce découplage montre comment la logique du jeu peut être modulée en plusieurs systèmes.

Note importante : comme indiqué dans le fichier `IMPORTANT_NOTE.txt`, le `DialogueComponent` est délibérément séparé du SDK pour illustrer la flexibilité de l'architecture. Si vous souhaitez intégrer un nouveau composant dans le moteur, vous devez l'ajouter dans `sdk/include/NovaEngine/ECS/Components.hpp` et gérer son chargement JSON dans la classe `Scene` ²⁶.

3.5 Système de caméra et viewport

NovaEngine utilise un `ViewportBackend` pour gérer la caméra et la conversion entre coordonnées monde et écran. La caméra suit généralement le joueur en centrant la vue sur sa position :

```
// Centre la caméra sur le joueur à chaque frame
VIEWPORT().setViewCenter(playerTransform->position);
```

Les positions et tailles des entités sont exprimées en **coordonnées monde** (ou pixels logiques). Le viewport se charge de convertir ces coordonnées en pixels de fenêtre en tenant compte de la résolution native et du letterboxing ²⁷. Par exemple, dans une résolution logique de 3840×2160, la position (1920, 1080) correspond au centre de l'écran. Si l'utilisateur joue en 1920×1080, ce centre sera converti en (960, 540) pixels réels ²⁸.

Lors du rendu, le viewport est appliqué automatiquement par le moteur, ce qui permet de dessiner les entités et l'UI sans se soucier de la résolution réelle. Pour déplacer la caméra manuellement ou créer un panoramique, vous pouvez modifier la vue via `VIEWPORT().setViewCenter()` ou `VIEWPORT().setZoom()`.

4 Interface utilisateur

Le système d'UI de NovaEngine est conçu pour être simple à utiliser et performant. Chaque élément UI dérive de `UIComponent` et possède sa propre logique de rendu et d'interaction ³. Le `UIManager` se charge de l'ordonnancement, de l'actualisation et de la propagation des événements ²⁹.

4.1 Créer une interface simple

Dans cet exemple, nous créons un menu avec un bouton « Commencer » et un texte d'en-tête :

```
class MenuScene : public Scene {
public:
    void onInitialize() override {
        // Crée un panel qui couvrira l'écran
        auto panel = std::make_shared<Panel>();
        panel->setSize({800, 600});
        panel->setBackgroundColor(Color{0, 0, 0, 150});

        // Ajoute un texte
        auto title = std::make_shared<Text>();
        title->setString("NovaEngine Demo");
        title->setPosition({200, 100});
        title->setFont(ResourceManager::getFont("monospace.ttf"));
        title->setCharacterSize(32);
        panel->addChild(title);

        // Ajoute un bouton
        auto startButton = std::make_shared<Button>();
        startButton->setText("Commencer");
        startButton->setPosition({300, 300});
        startButton->onClick = [this]() {
            // Transition vers une autre scène lorsque le bouton est cliqué
            SceneManager::loadScene("game");
        };
    };
};
```

```

        panel->addChild(startButton);

        // Insère le panel dans le UIManager
        uiManager->add(panel);
    }
};

```

Cet exemple illustre comment combiner plusieurs composants UI (Panel, Text, Button) au sein d'une scène. Le gestionnaire UI appelle automatiquement `onUpdate` et `onRender` pour chaque composant, et propage les événements de souris/clavier ²⁹.

4.2 Champ de texte et slider

Vous pouvez créer un champ de texte pour saisir le nom du joueur et un slider pour régler le volume. Voici un extrait :

```

auto nameInput = std::make_shared<TextInput>();
nameInput->setPosition({200, 200});
nameInput->setPlaceholder("Votre nom...");
panel->addChild(nameInput);

auto volumeSlider = std::make_shared<Slider>();
volumeSlider->setPosition({200, 250});
volumeSlider->setValue(0.5f);
volumeSlider->onValueChanged = [](float newValue) {
    // Met à jour le volume général
    AudioManager::setMasterVolume(newValue);
};
panel->addChild(volumeSlider);

```

Le `UIManager` gère la navigation au clavier, la tabulation et la sélection automatique des champs de texte. Chaque composant UI est dessiné dans l'ordre d'insertion.

5 Cœur applicatif et boucle principale

Pour créer un jeu, on dérive de la classe `Application` et on surcharge quelques méthodes :

```

class MyGame : public Application {
public:
    void onInitialize() override {
        // Initialise le moteur, charge la scène de départ
        SceneManager::initialize("assets/scenes/");
        SceneManager::loadScene("intro");
    }

    void onUpdate(float dt) override {
        // Mise à jour personnalisée avant les systèmes
        if (Input::isKeyPressed(Key::Escape)) {

```



```

        exit();
    }
}

void onRender() override {
    // Affichage supplémentaire (UI personnalisé)
}

void onEvent(const Event& event) override {
    // Gestion d'événements spécifiques
}
};

int main() {
    MyGame game;
    game.setWindowTitle("NovaEngine Demo");
    game.setWindowSize({1280, 720});
    game.run();
}

```

La méthode `run` inclut la boucle principale : elle récupère les événements de la fenêtre via le backend, met à jour la scène active et ses systèmes, puis dessine la scène et l'UI ⁴.

5.1 Exemple de flux de jeu

Le jeu RPG fourni avec NovaEngine suit un flux classique orchestré par la classe `Game`. Voici un résumé du déroulement inspiré de `ARCHITECTURE.md` ³⁰ :

1. **Initialisation** (`onInitialize`) :
2. Chargement du fichier de configuration via `ConfigManager` (dimensions de la fenêtre, mode plein écran, VSync, FPS limit, couleur de fond) ³¹.
3. Chargement de la police (Arial) et initialisation du `DialogueSystem`.
4. Initialisation du `SceneManager` et chargement des scènes JSON (`village.json`, `taverne.json`).
5. Création de l'entité joueur et enregistrement de son identifiant dans le `PlayerController` ³².
6. **Boucle de jeu** (`onUpdate`) :
7. Appel à `SceneManager::update()` pour mettre à jour la scène active et ses systèmes ³³.
8. Mise à jour du `PlayerController` : lecture des touches pour déplacer le joueur (WASD/Flèches), détection des NPCs dans un rayon donné, normalisation des mouvements diagonaux ³⁴.
9. Mise à jour de la caméra en centrant la vue sur la position du joueur ²⁷.
10. Mise à jour de l'UI via `UIManager::update()` ³⁵.

11. **Rendu** (`onRender`) :

12. Appel à `renderScene()` pour dessiner tous les sprites et effets, y compris l'indicateur « [E] » au-dessus des NPCs proches ³⁶ .

13. Affichage de la boîte de dialogue via `DialogueSystem::render()` si un dialogue est actif ³⁷ .

14. Rendu de l'UI via `UIManager::render()` .

15. **Événements** (`onEvent`) :

16. Touche **ESC** : ferme l'application ³⁸ .

17. Touche **E** : si un dialogue est actif, avance la ligne; sinon, démarre un dialogue avec le NPC le plus proche ³⁹ .

Cette organisation illustre comment structurer le code autour du moteur : les responsabilités sont séparées (contrôleur de joueur, système de dialogue, scène, UI), et la boucle de jeu appelle chacun de ces modules dans un ordre déterminé.

6 Gestion des ressources, définitions et scènes

6.1 Ressources (textures, sons, fonts)

Le moteur utilise un **ResourceManager** pour charger et mettre en cache les textures, sons, musiques et polices. Lorsqu'on demande une ressource via

`ResourceManager::getTexture("mon_image.png")` , elle est chargée la première fois puis réutilisée. Voici un exemple d'utilisation :

```
// Chargement d'une texture
sf::Texture* enemyTexture = ResourceManager::getTexture("enemy.png");

// Chargement d'une musique
sf::Music* backgroundMusic = ResourceManager::getMusic("ambient.ogg");
backgroundMusic->setLoop(true);
backgroundMusic->play();

// Chargement d'une police
sf::Font* font = ResourceManager::getFont("arial.ttf");
```

Les ressources sont identifiées par leur chemin relatif dans `assets/data/` et sont automatiquement mises en cache pour éviter de recharger plusieurs fois les mêmes fichiers.

6.2 Définitions de sprites et d'entités

Les **définitions** sont des fichiers JSON décrivant des sprites, animations, sons ou entités réutilisables. Elles se trouvent dans `assets/data/definitions/` et sont chargées via le `DefinitionManager`

⁵ . Un fichier `Sprites.json` peut par exemple définir plusieurs sprites génériques :

```

{
  "sprites": [
    {
      "id": "player",
      "texture": "data/sprites/player.png",
      "width": 64,
      "height": 64,
      "zOrder": 0
    },
    {
      "id": "npc_merchant",
      "texture": "data/sprites/npc_merchant.png",
      "width": 64,
      "height": 64
    }
  ]
}

```

Chaque entrée possède un identifiant unique (`id`) et pointe vers un fichier PNG. Les champs optionnels incluent :

- `width` / `height` : taille du sprite en pixels logiques ⁴⁰ ;
- `scale` : facteur d'échelle uniforme appliqué à la définition ⁴¹ ;
- `zOrder` : ordre de dessin (plus grand = devant) ⁴² ;
- `textureRect` : rectangle `[x, y, width, height]` pour découper une sprite sheet ⁴³ ;
- `origin` : point d'ancrage `[x, y]`, centre par défaut ⁴⁴ .

Les définitions sont chargées une seule fois au démarrage, puis référencées par les scènes via l'identifiant `id` ⁵ . Cette approche évite la duplication de données et permet de modifier l'apparence d'un sprite à un seul endroit.

6.3 Format des scènes JSON

Une scène se décrit dans un fichier JSON contenant un nom, une couleur de fond et une liste d'entités. Voici le format minimal d'une scène :

```

{
  "name": "Test Scene",
  "backgroundColor": [30, 30, 40, 255],
  "entities": [
    {
      "type": "sprite",
      "spriteID": "player",
      "x": 1920,
      "y": 1080
    }
  ]
}

```

Chaque entité dans la scène possède les champs suivants :

- `type` : le type d'entité à créer (par exemple `"sprite"`, `"light"`, `"audio"`), qui détermine quel composant principal sera ajouté ⁴⁵ ;
- `spriteID` : identifiant d'un sprite défini dans `Sprites.json` ⁴⁵ ;
- `x`, `y` : position en **pixels logiques** dans la résolution native (par défaut 3840×2160) ⁴⁶ ;
- Champs optionnels : `scale` pour agrandir/réduire le sprite, `rotation` en degrés, `zOrder` pour modifier l'ordre de rendu, `origin` pour changer le point d'ancrage ⁴⁵ ⁴⁷ .

Le format supporte également un champ `position: [x, y]` pour compatibilité avec d'anciens fichiers ⁴⁸ . Les couleurs de fond peuvent être ajustées pour créer différentes ambiances : un ciel bleu clair `[135, 206, 235, 255]` pour un village en journée ou un bleu foncé `[20, 20, 40, 255]` pour une scène de nuit ⁹ .

Les positions et tailles utilisent la **résolution logique** définie dans le fichier de configuration (`nativeWidth` × `nativeHeight`). Le moteur met automatiquement à l'échelle et ajoute des bandes noires (letterboxing) pour s'adapter à la résolution réelle de la fenêtre ⁴⁹ .

Pour charger et activer une scène, on appelle :

```
SceneManager::loadScene("data/scenes/village.json", "village");
SceneManager::loadScene("data/scenes/taverne.json", "taverne");
SceneManager::setActiveScene("village");
```

Le `SceneManager` charge toutes les entités définies et applique leurs composants. On peut ensuite basculer d'une scène à l'autre en changeant la scène active ⁶ .

6.4 Création et gestion des sprites

Pour les projets d'exemple, NovaEngine fournit un script Python `create_sprites.py` qui génère automatiquement des sprites de test (carrés 64×64) pour le joueur et les NPCs ⁵⁰ . Il suffit d'exécuter :

```
python create_sprites.py
```

Cela crée trois images : `player.png` (bleu clair), `npc_merchant.png` (vert) et `npc_innkeeper.png` (orange) dans le dossier `assets/data/`. Vous pouvez aussi créer vos propres sprites avec un éditeur d'image en respectant la taille 64×64 ⁵¹ . Des couleurs recommandées sont :

- Joueur : bleu clair (100, 200, 255)
- Marchand : vert (50, 200, 100)
- Tavernier : orange (255, 150, 50)

Assurez-vous également de copier une police TrueType (par exemple `arial.ttf`) dans `assets/data/` pour l'affichage du texte ⁵² .

Autres méthodes de création de sprites et conseils

Vous pouvez également créer manuellement les sprites avec des outils simples ou réutiliser des sprites existants :

- **Paint (Windows)** : ouvrez l'application `mspaint`, créez une nouvelle image 64×64, remplissez-la de la couleur désirée et enregistrez au format PNG dans `assets/data/` ⁵¹.
- **GIMP** (gratuit et multiplateforme) : créez une nouvelle image 64×64, dessinez votre sprite, puis exportez-le au format PNG ⁵³.
- **Sprites existants** : n'importe quel PNG 64×64 provenant de sites comme OpenGameArt.org, itch.io ou Kenney.nl peut être utilisé directement ⁵⁴.

Lors de la préparation des ressources, vérifiez que les fichiers `player.png`, `npc_merchant.png`, `npc_innkeeper.png` et `arial.ttf` sont bien présents dans `assets/data/` ⁵⁵. Toute police TrueType peut être renommée en `arial.ttf` pour être chargée par le moteur ⁵⁶.

Pour les artistes plus ambitieux, vous pouvez :

- ajouter des détails et de la transparence grâce au canal alpha,
- créer plusieurs frames pour des animations futures,
- utiliser des logiciels spécialisés comme **Aseprite**, **Piskel** ou **GraphicsGale** pour un travail de pixel art ⁵⁷.

Si le moteur affiche un message « **Failed to load texture** », vérifiez l'extension (.png), la casse du nom de fichier et les permissions ; ajustez la taille (`size`) dans le JSON si les sprites semblent trop grands ou trop petits ⁵⁸.

7 Post-traitements et shaders

Pour ajouter des effets visuels, on utilise le `PostProcessPipeline`. On instancie les effets et on les ajoute à la scène. Chaque effet dérive de `PostProcessEffect` et possède des paramètres réglables. Voici comment configurer plusieurs effets :

```
// Initialisation dans onInitialize()
auto pipeline = std::make_shared<PostProcessPipeline>();

// Ajout d'un effet CRT
auto crt = std::make_shared<CRTEffect>();
crt->setStrength(0.8f);
pipeline->addEffect(crt);

// Ajout d'un effet Bloom
auto bloom = std::make_shared<BloomEffect>();
bloom->setThreshold(1.0f);
bloom->setIntensity(0.5f);
pipeline->addEffect(bloom);

// Ajout de l'occlusion ambiante
auto ssao = std::make_shared<SSAOEffect>();
ssao->setRadius(4.0f);
```

```

ssao->setStrength(1.2f);
pipeline->addEffect(ssao);

// Lier le pipeline à la scène
scene->setPostProcessPipeline(pipeline);

// Plus tard, vous pouvez activer/désactiver un effet selon un input
if (Input::isKeyPressed(Key::F1)) {
    crt->toggleEnabled();
}

```

La documentation officielle décrit le fonctionnement interne du pipeline : les effets sont appliqués dans l'ordre d'insertion, chaque effet peut posséder ses propres textures intermédiaires et paramètres, et on peut créer des effets personnalisés en dérivant de `PostProcessEffect` ¹⁰.

7.1 Créer un effet personnalisé

Pour ajouter un effet sur mesure, il faut créer une classe dérivant de `PostProcessEffect` avec quatre méthodes : `initialize`, `shutdown`, `apply` et `isEnabledByDefault`. On associe ensuite un shader GLSL. Voici un exemple minimal d'effet qui inverse les couleurs (inspiré de la documentation) :

```

class InvertColorEffect : public PostProcessEffect {
public:
    bool initialize() override {
        shader = ResourceManager::loadShader("invert.frag");
        return shader != nullptr;
    }
    void shutdown() override {}
    void apply(RenderTexture& sceneTexture, RenderTarget& finalTarget)
    override {
        // Passe la texture de la scène au shader et dessine un quad
        shader->setUniform("u_scene", sceneTexture.getTexture());
        GRAPHICS().drawFullScreenQuad(finalTarget, shader);
    }
    bool isEnabledByDefault() const override { return true; }
private:
    Shader* shader;
};

// Ajout de l'effet personnalisé
auto invert = std::make_shared<InvertColorEffect>();
pipeline->addEffect(invert);

```

Le shader `invert.frag` pourrait être :

```

uniform sampler2D u_scene;
void main() {
    vec4 color = texture(u_scene, gl_FragCoord.xy / resolution);
}

```

```
gl_FragColor = vec4(1.0 - color.rgb, color.a);
}
```

Cette approche est conforme à la description du processus de création d'un effet dans la documentation ¹⁰.

8 Gestion des événements

Le système d'événements se base sur une structure `Event` et un `EventDispatcher`. Les événements sont utilisés par les systèmes d'entrée, d'UI et d'activateur pour communiquer de manière découplée. Un gestionnaire peut s'inscrire à des types d'événements spécifiques et recevoir les événements pertinents. Par exemple :

```
// Écouteur pour un événement de porte
EventDispatcher::subscribe("open_door", [](const Event& event) {
    // Ouvre la porte associée à l'événement
    auto doorId = event.getData<int>("doorId");
    // Exécuter la logique...
});

// Émission de l'événement depuis l'ActivatorSystem
Event evt("open_door");
evt.setData("doorId", currentDoorId);
EventDispatcher::emit(evt);
```

Le système d'événements facilite la communication entre différentes parties du moteur sans créer de dépendances directes.

9 Patterns et bonnes pratiques

NovaEngine utilise plusieurs patrons de conception afin d'assurer modularité et maintenabilité :

Singleton pour le gestionnaire de backend, **Facade** pour simplifier l'accès aux backends, **Strategy** pour les systèmes de rendu et de post-traitement, **Observer** pour le système d'événements et **Factory** pour la création d'entités à partir de définitions. Il est recommandé :

- d'utiliser les définitions JSON pour séparer les données du code (data driven design) ⁵⁹ ;
- de créer ses propres systèmes plutôt que de surcharger les systèmes existants ;
- de profiter du pipeline de post-traitement pour créer des effets personnalisés selon vos besoins;
- de séparer la logique de jeu (ECS) de l'UI et du rendu pour garder un code clair.

10 Fonctionnalités non démontrées et extensions suggérées

Le moteur offre de nombreuses fonctionnalités qui ne sont pas activées dans l'exemple **Simple RPG** mais qui sont pleinement disponibles dans le SDK :

- **Animations** à base de sprite sheets via `AnimationComponent`.
- **Lumières 2D** et ombres dynamiques.
- **Effets audio** avec `AudioComponent` pour les bruitages et musiques.

- **Collisions physiques** gérées par `PhysicsSystem`.
- **Système de particules** pour les effets visuels (explosions, étincelles).
- **Interface utilisateur avancée** avec des sliders, champs de texte et panneaux complexes.
- **Sauvegarde/chargement** de l'état du jeu via sérialisation JSON.
- **Navigation multi-scène des NPCs** et pathfinding global via le `JourneySystem`.
- **Inventaire et quêtes** grâce à de nouveaux composants personnalisés.

Pour enrichir votre jeu, voici quelques pistes d'extensions inspirées de la documentation ⁶⁰ :

- **Ajouter plus de NPCs** et de dialogues en dupliquant les entités existantes.
- **Créer de nouvelles scènes** (maison, boutique) et les relier dans `scenegraph.json`.
- **Modifier l'ambiance** en changeant `backgroundColor` ou en ajoutant des effets de post-traitement (bloom, SSAO, color grading).
- **Implémenter un inventaire** ou un système de combat/quête en ajoutant des composants et systèmes dédiés.
- **Ajouter des transitions de scène** en utilisant `SceneTransitionComponent` pour téléporter le joueur entre scènes.
- **Animer le sprite du joueur** ou des NPCs avec des animations de marche.

Ces suggestions démontrent comment exploiter la modularité du modèle ECS et les fichiers JSON pour étendre le gameplay sans alourdir le cœur du moteur.

Conclusion

Cette documentation complète présente le fonctionnement interne de NovaEngine et des exemples de code pour tous les modules principaux. En respectant l'architecture ECS, en exploitant le système d'UI et de post-traitement et en suivant les bonnes pratiques décrites, vous disposerez d'une base solide pour développer vos propres jeux 2D. N'hésitez pas à explorer la documentation originale pour plus de détails sur les fonctionnalités avancées et à consulter les exemples fournis dans le dépôt.

Annexe : Structure des fichiers et guides

L'arborescence du dépôt NovaEngine contient plusieurs répertoires et documents importants. Voici une vue d'ensemble des principaux dossiers et fichiers avec leur rôle :

```

Nova/
├── NOVAENGINE_DOCUMENTATION.md    # Document technique complet (cité tout au
long de cette doc)
├── README.md                      # Brève description du projet
├── client/                         # Code de l'exemple RPG complet
│   ├── main.cpp                   # Point d'entrée de l'exemple client
│   └── ARCHITECTURE.md            # Description détaillée des modules (Game,
DialogueSystem, PlayerController) 61
│   ├── src/                       # Modules spécifiques à l'exemple
│   │   ├── Game.cpp               # Classe principale orchestrant le jeu 62
│   │   └── Dialogue/              # Composant et système de dialogue
personnalisés 63
│   └── Player/                    # Contrôleur du joueur (détection,
mouvement) 64
└── assets/                        # Ressources pour l'exemple

```



```

|       └─ data/
|           └─ sprites/           # Images PNG pour le joueur et les NPCs 65
|           └─ fonts/            # Polices .ttf (ex : Arial) 52
|           └─ definitions/      # Définitions de sprites, NPCs, animations
5
|       └─ scenes/              # Scènes JSON (village, taverne, etc.) 66
|       └─ scenegraph.json      # Graphe reliant les scènes entre elles 66
└─ server/                     # Exemple de serveur réseau très basique
    └─ main.cpp                # Écoute un port et diffuse les paquets
└─ examples/                  # Autres exemples de jeux
    └─ simple_rpg/            # Mini jeu démontrant les fonctionnalités
de base
|       └─ README.md           # Présentation et contrôles de l'exemple
67
|       └─ FEATURES.md         # Détails des fonctionnalités utilisées
(ECS, scènes, dialogue) 68
|       └─ CREATE_SPRITES.md   # Guide pour générer des sprites de test
50
|       └─ IMPORTANT_NOTE.txt   # Avertissement sur DialogueComponent hors
SDK 69
|       └─ main.cpp            # Point d'entrée du mini-jeu
|       └─ assets/data/        # Ressources spécifiques à l'exemple
(sprites, definitions, scènes)
|       └─ compile.bat         # Script de compilation Windows
└─ sdk/                        # Code source du moteur et dépendances
    └─ include/NovaEngine/     # Headers publics du moteur (ECS, backend,
UI...)
        └─ include/SFML/      # Headers de SFML fournis pour le backend
        └─ libs/              # Bibliothèques SFML statiques
        └─ MinGW/             # Toolchain Windows et modules CMake
        └─ src/               # Implémentation du moteur (C++)
└─ tools/ ou scripts divers   # Scripts utilitaires, dépendances
supplémentaires

```

Documents complémentaires

- **ARCHITECTURE.md** : décrit la structure des modules du jeu RPG et le flux de données entre Game, DialogueSystem et PlayerController. Ce document explique comment organiser son code dans un projet réel 70 .
- **FEATURES.md** : détaille les fonctionnalités démontrées dans l'exemple Simple RPG : composants utilisés, chargement de scènes, architecture deux niveaux, système de dialogue, caméra, événements et ressources 68 71 .
- **CREATE_SPRITES.md** : guide pour créer rapidement des sprites de test à l'aide d'un script Python ou d'éditeurs d'images 50 .
- **SCENE_SYSTEM.md** : résumé du système de scène utilisé dans la version compilée du client, avec format simplifié des fichiers de définitions et de scènes 72 .
- **IMPORTANT_NOTE.txt** : avertit que le DialogueComponent est défini localement dans l'exemple et n'est pas fourni par le SDK principal 69 .

Compilation et contrôles par défaut

Pour compiler le client de démonstration sous Windows, exécutez le script `client_compile.bat` situé à la racine du dépôt. Ce script compile automatiquement `Game.cpp`, `DialogueSystem.cpp`, `PlayerController.cpp` et les autres sources du moteur, puis génère l'exécutable dans `client/bin/Release/` ⁷³.

Les contrôles par défaut du jeu RPG sont :

- **W, A, S, D** ou les flèches : déplacer le personnage ⁷⁴ ;
- **E** : interagir avec un NPC (démarrer/avancer un dialogue) ⁷⁵ ;
- **ESC** : quitter le jeu ³⁸.

Bonnes pratiques et performance

Le fichier `ARCHITECTURE.md` souligne l'importance de la **modularité** : chaque système doit résider dans son propre module et `Game.cpp` joue le rôle d'orchestre central ⁷⁶. Il est également recommandé d'externaliser les paramètres (taille de fenêtre, FPS, etc.) dans un fichier `config.json` et d'organiser les assets par type (sprites, polices, définitions, scènes) ³¹.

Pour optimiser les performances :

- Les entités sont parcourues une seule fois par frame par chaque système ⁷⁷ ;
- Le calcul du NPC le plus proche est effectué une fois par frame, pas à chaque pression de touche ⁷⁸ ;
- Les ressources (textures, fonts) sont chargées une seule fois et réutilisées via des handles ⁷⁹.

Appliquer ces pratiques vous aidera à maintenir un code clair et un jeu fluide même en ajoutant des fonctionnalités avancées.

Ces documents complémentaires peuvent être consultés pour approfondir certains aspects spécifiques du moteur ou pour comprendre comment l'exemple Simple RPG est organisé. Ils restent d'actualité si vous utilisez la branche `main` du dépôt.

1 2 3 4 7 8 10 11 12 13 14 15 16 17 18 19 20 21 22 29 59

NOVAENGINE_DOCUMENTATION.md

https://github.com/f0xnet/Nova/blob/main/NOVAENGINE_DOCUMENTATION.md

5 6 24 27 60 65 66 68 71 FEATURES.md

https://github.com/f0xnet/Nova/blob/main/examples/simple_rpg/FEATURES.md

9 52 67 README.md

https://github.com/f0xnet/Nova/blob/main/examples/simple_rpg/README.md

23 26 69 IMPORTANT_NOTE.txt

https://github.com/f0xnet/Nova/blob/main/examples/simple_rpg/IMPORTANT_NOTE.txt

25 30 31 32 33 34 35 36 37 38 39 61 62 63 64 70 73 74 75 76 77 78 79 ARCHITECTURE.md

<https://github.com/f0xnet/Nova/blob/main/client/ARCHITECTURE.md>

28 40 41 42 43 44 45 46 47 48 49 72 SCENE_SYSTEM.md

https://github.com/f0xnet/Nova/blob/main/client/bin/Release/data/SCENE_SYSTEM.md

50 51 53 54 55 56 57 58 CREATE_SPRITES.md

https://github.com/f0xnet/Nova/blob/main/examples/simple_rpg/CREATE_SPRITES.md